

Week 3 Assignment

SQL & Excel for Data Analytics

Student Name	[Your Name]
Roll No. / ID	[Your ID]
Course	Data Analytics
Marks	100
Tools Used	SQLite (SQL), Microsoft Excel
Google Drive Link	[Paste your shared Drive link here]

Contents

Task 1 Databases & SELECT • Task 2 WHERE, ORDER BY & Aggregates • Task 3 GROUP BY & HAVING • Task 4 Joins • Task 5 Subqueries • Task 6 Formatting, Sorting & Filtering • Task 7 Conditional Formatting & Functions • Task 8 VLOOKUP & XLOOKUP • Task 9 Pivot Table & Charts

Task 1: Introduction to Databases & SELECT Statement

A **database** is an organised collection of data stored in **tables** (rows = records, columns = fields). A **relational database** links tables using keys: a **primary key** uniquely identifies each row and a **foreign key** refers to the primary key of another table. SQL (Structured Query Language) is used to create, read, update and analyse this data.

Sample database used (company data)

departments(dept_id PK, dept_name, location) | **employees**(emp_id PK, first_name, last_name, dept_id FK, job_title, salary, hire_date, city) | **products**(product_id PK, product_name, category, price, stock). It has 20 employees, 6 departments and 10 products. One employee has no department and one department (Legal) has no employees, so the joins in Task 4 show real differences.

```
CREATE TABLE departments (  
    dept_id    INTEGER PRIMARY KEY,  
    dept_name  VARCHAR(50) NOT NULL,  
    location   VARCHAR(50)  
);  
  
CREATE TABLE employees (  
    emp_id     INTEGER PRIMARY KEY,  
    first_name VARCHAR(30) NOT NULL,  
    last_name  VARCHAR(30) NOT NULL,  
    dept_id    INTEGER,           -- foreign key -> departments.dept_id  
    job_title  VARCHAR(50),  
    salary     INTEGER,  
    hire_date  DATE,  
    city       VARCHAR(30),  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);  
  
CREATE TABLE products (  
    product_id  INTEGER PRIMARY KEY,  
    product_name VARCHAR(50),  
    category    VARCHAR(30),  
    price       INTEGER,  
    stock       INTEGER  
);
```

1.1 Display all records

```
SELECT * FROM employees;
```

Output (20 rows)

emp_id	first_name	last_name	dept_id	job_title	salary	hire_date	city
101	Aarav	Sharma	2	Software Engineer	65000	2021-03-15	Noida
102	Priya	Verma	1	Sales Executive	42000	2020-07-01	Lucknow
103	Rohan	Gupta	2	Data Analyst	58000	2022-01-10	Noida
104	Sneha	Singh	3	HR Manager	72000	2019-05-20	Kanpur
105	Vikram	Yadav	4	Accountant	48000	2021-09-12	Delhi
106	Anjali	Mishra	5	Marketing Lead	61000	2020-11-30	Agra
107	Karan	Tiwari	2	DevOps Engineer	70000	2018-02-25	Noida
108	Neha	Pandey	1	Sales Manager	80000	2017-06-18	Lucknow
109	Amit	Kumar	4	Financial Analyst	55000	2022-08-05	Delhi
110	Pooja	Chauhan	3	HR Executive	38000	2023-01-16	Kanpur
111	Rahul	Saxena	2	Data Scientist	85000	2019-10-09	Noida
112	Simran	Kaur	5	Content Writer	35000	2023-04-03	Agra
113	Deepak	Joshi	1	Sales Executive	40000	2021-12-01	Lucknow
114	Meera	Nair	4	Finance Manager	90000	2016-03-14	Delhi

emp_id	first_name	last_name	dept_id	job_title	salary	hire_date	city
115	Arjun	Mehta	2	Software Engineer	62000	2022-06-21	Noida
116	Kavya	Rao	5	SEO Specialist	45000	2021-05-11	Agra
117	Sanjay	Dubey	1	Sales Executive	43000	2020-02-17	Lucknow
118	Ritu	Agarwal	3	Recruiter	41000	2022-11-28	Kanpur
119	Manish	Bajpai	2	QA Engineer	52000	2021-07-19	Noida
120	Tanya	Kapoor	NULL	Intern	15000	2024-01-08	Delhi

Explanation: SELECT * returns every column and every row of the employees table.

1.2 Select specific columns

```
SELECT first_name, last_name, job_title, salary
FROM employees;
```

Output (20 rows)

first_name	last_name	job_title	salary
Aarav	Sharma	Software Engineer	65000
Priya	Verma	Sales Executive	42000
Rohan	Gupta	Data Analyst	58000
Sneha	Singh	HR Manager	72000
Vikram	Yadav	Accountant	48000
Anjali	Mishra	Marketing Lead	61000
Karan	Tiwari	DevOps Engineer	70000
Neha	Pandey	Sales Manager	80000
Amit	Kumar	Financial Analyst	55000
Pooja	Chauhan	HR Executive	38000
Rahul	Saxena	Data Scientist	85000
Simran	Kaur	Content Writer	35000
Deepak	Joshi	Sales Executive	40000
Meera	Nair	Finance Manager	90000
Arjun	Mehta	Software Engineer	62000
Kavya	Rao	SEO Specialist	45000
Sanjay	Dubey	Sales Executive	43000
Ritu	Agarwal	Recruiter	41000
Manish	Bajpai	QA Engineer	52000
Tanya	Kapoor	Intern	15000

Explanation: Only the four named columns are returned instead of the full table.

1.3 Column aliases

```
SELECT first_name || ' ' || last_name AS full_name,
       salary AS monthly_salary,
       salary * 12 AS annual_salary
FROM employees;
```

Output (20 rows)

full_name	monthly_salary	annual_salary
Aarav Sharma	65000	780000
Priya Verma	42000	504000
Rohan Gupta	58000	696000

full_name	monthly_salary	annual_salary
Sneha Singh	72000	864000
Vikram Yadav	48000	576000
Anjali Mishra	61000	732000
Karan Tiwari	70000	840000
Neha Pandey	80000	960000
Amit Kumar	55000	660000
Pooja Chauhan	38000	456000
Rahul Saxena	85000	1020000
Simran Kaur	35000	420000
Deepak Joshi	40000	480000
Meera Nair	90000	1080000
Arjun Mehta	62000	744000
Kavya Rao	45000	540000
Sanjay Dubey	43000	516000
Ritu Agarwal	41000	492000
Manish Bajpai	52000	624000
Tanya Kapoor	15000	180000

Explanation: AS gives readable names to columns/expressions. || joins text (use CONCAT() in MySQL).

Task 2: WHERE, ORDER BY & Aggregate Functions

2.1 WHERE with comparison operator (>)

```
SELECT emp_id, first_name, job_title, salary
FROM employees
WHERE salary > 60000;
```

Output (8 rows)

emp_id	first_name	job_title	salary
101	Aarav	Software Engineer	65000
104	Sneha	HR Manager	72000
106	Anjali	Marketing Lead	61000
107	Karan	DevOps Engineer	70000
108	Neha	Sales Manager	80000
111	Rahul	Data Scientist	85000
114	Meera	Finance Manager	90000
115	Arjun	Software Engineer	62000

Explanation: Returns only employees whose salary is greater than 60,000.

2.2 WHERE with multiple conditions (AND / =)

```
SELECT emp_id, first_name, city, salary
FROM employees
WHERE city = 'Noida' AND salary >= 60000;
```

Output (4 rows)

emp_id	first_name	city	salary
101	Aarav	Noida	65000
107	Karan	Noida	70000
111	Rahul	Noida	85000
115	Arjun	Noida	62000

Explanation: AND requires both conditions to be true.

2.3 WHERE with BETWEEN and <>

```
SELECT emp_id, first_name, salary, city
FROM employees
WHERE salary BETWEEN 40000 AND 50000
AND city <> 'Agra';
```

Output (5 rows)

emp_id	first_name	salary	city
102	Priya	42000	Lucknow
105	Vikram	48000	Delhi
113	Deepak	40000	Lucknow
117	Sanjay	43000	Lucknow
118	Ritu	41000	Kanpur

Explanation: BETWEEN is inclusive; <> means 'not equal to'.

2.4 ORDER BY (descending)

```
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

Output (20 rows)

first_name	last_name	salary
Meera	Nair	90000
Rahul	Saxena	85000
Neha	Pandey	80000
Sneha	Singh	72000
Karan	Tiwari	70000
Aarav	Sharma	65000
Arjun	Mehta	62000
Anjali	Mishra	61000
Rohan	Gupta	58000
Amit	Kumar	55000
Manish	Bajpai	52000
Vikram	Yadav	48000
Kavya	Rao	45000
Sanjay	Dubey	43000
Priya	Verma	42000
Ritu	Agarwal	41000
Deepak	Joshi	40000
Pooja	Chauhan	38000
Simran	Kaur	35000
Tanya	Kapoor	15000

Explanation: Sorts employees from highest to lowest salary.

2.5 ORDER BY on two columns

```
SELECT city, first_name, salary
FROM employees
ORDER BY city ASC, salary DESC;
```

Output (20 rows)

city	first_name	salary
Agra	Anjali	61000
Agra	Kavya	45000
Agra	Simran	35000
Delhi	Meera	90000
Delhi	Amit	55000
Delhi	Vikram	48000
Delhi	Tanya	15000
Kanpur	Sneha	72000
Kanpur	Ritu	41000
Kanpur	Pooja	38000
Lucknow	Neha	80000
Lucknow	Sanjay	43000
Lucknow	Priya	42000

city	first_name	salary
Lucknow	Deepak	40000
Noida	Rahul	85000
Noida	Karan	70000
Noida	Aarav	65000
Noida	Arjun	62000
Noida	Rohan	58000
Noida	Manish	52000

Explanation: Sorts by city A-Z first, then by salary high-to-low within each city.

2.6 Aggregate functions

```
SELECT COUNT(*) AS total_employees,
       SUM(salary) AS total_salary,
       ROUND(AVG(salary),2) AS average_salary,
       MIN(salary) AS lowest_salary,
       MAX(salary) AS highest_salary
FROM employees;
```

Output (1 row)

total_employees	total_salary	average_salary	lowest_salary	highest_salary
20	1097000	54850.0	15000	90000

Explanation: COUNT, SUM, AVG, MIN and MAX summarise the whole table into one row.

Task 3: GROUP BY & HAVING

3.1 GROUP BY with aggregates

```
SELECT dept_id,  
       COUNT(*)      AS num_employees,  
       SUM(salary)    AS total_salary,  
       ROUND(AVG(salary),0) AS avg_salary,  
       MAX(salary)    AS max_salary  
FROM employees  
GROUP BY dept_id  
ORDER BY dept_id;
```

Output (6 rows)

dept_id	num_employees	total_salary	avg_salary	max_salary
NULL	1	15000	15000.0	15000
1	4	205000	51250.0	80000
2	6	392000	65333.0	85000
3	3	151000	50333.0	72000
4	3	193000	64333.0	90000
5	3	141000	47000.0	61000

Explanation: One row per department. The NULL group is the intern who has no department.

3.2 GROUP BY city

```
SELECT city, COUNT(*) AS num_employees, ROUND(AVG(salary),0) AS avg_salary  
FROM employees  
GROUP BY city  
ORDER BY num_employees DESC;
```

Output (5 rows)

city	num_employees	avg_salary
Noida	6	65333.0
Lucknow	4	51250.0
Delhi	4	52000.0
Kanpur	3	50333.0
Agra	3	47000.0

Explanation: Shows head-count and average pay for each city.

3.3 HAVING - filter groups

```
SELECT dept_id, COUNT(*) AS num_employees, ROUND(AVG(salary),0) AS avg_salary  
FROM employees  
GROUP BY dept_id  
HAVING AVG(salary) > 55000;
```

Output (2 rows)

dept_id	num_employees	avg_salary
2	6	65333.0
4	3	64333.0

Explanation: HAVING filters after grouping (WHERE filters before). Only departments with average salary above 55,000 remain.

3.4 WHERE + GROUP BY + HAVING together

```
SELECT dept_id, COUNT(*) AS num_employees, SUM(salary) AS total_salary
FROM employees
WHERE salary > 40000
GROUP BY dept_id
HAVING COUNT(*) >= 3;
```

Output (3 rows)

dept_id	num_employees	total_salary
1	3	165000
2	6	392000
4	3	193000

Explanation: WHERE first removes low salaries, rows are grouped, then HAVING keeps departments with 3 or more remaining employees.

Task 4: SQL Joins

INNER JOIN keeps only matching rows from both tables. **LEFT JOIN** keeps all rows of the left table plus matches from the right (NULL where none). **RIGHT JOIN** keeps all rows of the right table plus matches from the left. Common column: dept_id.

4.1 INNER JOIN

```
SELECT e.emp_id, e.first_name, e.job_title, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Output (19 rows)

emp_id	first_name	job_title	dept_name	location
101	Aarav	Software Engineer	IT	Noida
102	Priya	Sales Executive	Sales	Lucknow
103	Rohan	Data Analyst	IT	Noida
104	Sneha	HR Manager	HR	Kanpur
105	Vikram	Accountant	Finance	Delhi
106	Anjali	Marketing Lead	Marketing	Agra
107	Karan	DevOps Engineer	IT	Noida
108	Neha	Sales Manager	Sales	Lucknow
109	Amit	Financial Analyst	Finance	Delhi
110	Pooja	HR Executive	HR	Kanpur
111	Rahul	Data Scientist	IT	Noida
112	Simran	Content Writer	Marketing	Agra
113	Deepak	Sales Executive	Sales	Lucknow
114	Meera	Finance Manager	Finance	Delhi
115	Arjun	Software Engineer	IT	Noida
116	Kavya	SEO Specialist	Marketing	Agra
117	Sanjay	Sales Executive	Sales	Lucknow
118	Ritu	Recruiter	HR	Kanpur
119	Manish	QA Engineer	IT	Noida

Explanation: Returns only rows that match in BOTH tables. Tanya (no dept) and Legal (no employees) are excluded.

4.2 LEFT JOIN

```
SELECT e.emp_id, e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id >= 118;
```

Output (3 rows)

emp_id	first_name	dept_name
118	Ritu	HR
119	Manish	IT
120	Tanya	NULL

Explanation: Keeps ALL employees (left table). Tanya appears with a NULL department. (Filtered to last few rows for a short output.)

4.3 RIGHT JOIN

```
SELECT d.dept_id, d.dept_name, e.first_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IN (3, 6)
ORDER BY d.dept_id;
```

Output (4 rows)

dept_id	dept_name	first_name
3	HR	Sneha
3	HR	Pooja
3	HR	Ritu
6	Legal	NULL

Explanation: Keeps ALL departments (right table). Legal appears with a NULL employee because nobody works there. (Filtered to HR and Legal.)

4.4 JOIN + GROUP BY (bonus)

```
SELECT d.dept_name, COUNT(e.emp_id) AS num_employees, ROUND(AVG(e.salary),0) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY num_employees DESC;
```

Output (6 rows)

dept_name	num_employees	avg_salary
IT	6	65333.0
Sales	4	51250.0
Marketing	3	47000.0
HR	3	50333.0
Finance	3	64333.0
Legal	0	NULL

Explanation: Combines a join with grouping. Legal shows 0 employees because of the LEFT JOIN.

Task 5: SQL Subqueries

5.1 Employees earning more than the average salary

```
SELECT emp_id, first_name, job_title, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

Output (10 rows)

emp_id	first_name	job_title	salary
114	Meera	Finance Manager	90000
111	Rahul	Data Scientist	85000
108	Neha	Sales Manager	80000
104	Sneha	HR Manager	72000
107	Karan	DevOps Engineer	70000
101	Aarav	Software Engineer	65000
115	Arjun	Software Engineer	62000
106	Anjali	Marketing Lead	61000
103	Rohan	Data Analyst	58000
109	Amit	Financial Analyst	55000

Explanation: The inner query computes the average salary (single value); the outer query compares each salary to it.

5.2 Products priced higher than the average product price

```
SELECT product_name, category, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Output (3 rows)

product_name	category	price
Laptop	Electronics	55000
Tablet	Electronics	30000
Smartphone	Electronics	22000

Explanation: Same idea applied to the products table.

5.3 Subquery with IN (employees in Delhi-based departments)

```
SELECT first_name, job_title
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Delhi');
```

Output (3 rows)

first_name	job_title
Vikram	Accountant
Amit	Financial Analyst
Meera	Finance Manager

Explanation: The inner query returns a list of dept_ids; IN checks membership in that list.

5.4 Highest-paid employee in each department (correlated subquery)

```
SELECT e.first_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary = (SELECT MAX(salary) FROM employees WHERE dept_id = e.dept_id)
ORDER BY e.dept_id;
```

Output (5 rows)

first_name	dept_id	salary
Neha	1	80000
Rahul	2	85000
Sneha	3	72000
Meera	4	90000
Anjali	5	61000

Explanation: A correlated subquery runs once per outer row, using that row's dept_id.

Excel Tasks - Overview

All Excel work is in **Week3_Excel_Workbook.xlsx**, one sheet per task (Task6_Formatted, Task6_Filtered, Task7_CondFmt_Functions, Task8_Lookups, Task9_SalesData, Task9_Pivot_Chart). The employee dataset is the same data used in the SQL tasks, so results can be cross-checked (e.g. total salary = 10,97,000 in both SQL and Excel).

Task 6: Data Formatting, Sorting & Filtering

Formatting: bold white headers on a dark-blue fill, borders, banded rows, salary in Rs. currency format, dates as DD-MMM-YYYY, centred IDs/ratings, frozen header row. **Sorting:** Department A-Z, then Salary high-to-low (Data > Sort). **Filtering:** AutoFilter enabled on the header row.

Task 6 - Employee Dataset: Formatted, Sorted (Dept A-Z, Salary high-to-low)

Data sorted by Department (A-Z) then Salary (descending). AutoFilter buttons are on the header row.

Emp ID	Employee Name	Department	City	Salary (INR)	Join Date	Rating (1-5)	Job Title
114	Meera Nair	Finance	Delhi	₹90,000	14-Mar-2016	4	Finance Manager
109	Amit Kumar	Finance	Delhi	₹55,000	05-Aug-2022	3	Financial Analyst
105	Vikram Yadav	Finance	Delhi	₹48,000	12-Sep-2021	3	Accountant
104	Sneha Singh	HR	Kanpur	₹72,000	20-May-2019	4	HR Manager
118	Ritu Agarwal	HR	Kanpur	₹41,000	28-Nov-2022	4	Recruiter
110	Pooja Chauhan	HR	Kanpur	₹38,000	16-Jan-2023	2	HR Executive
111	Rahul Saxena	IT	Noida	₹85,000	09-Oct-2019	5	Data Scientist
107	Karan Tiwari	IT	Noida	₹70,000	25-Feb-2018	5	DevOps Engineer
101	Aarav Sharma	IT	Noida	₹65,000	15-Mar-2021	4	Software Engineer
115	Arjun Mehta	IT	Noida	₹62,000	21-Jun-2022	4	Software Engineer
103	Rohan Gupta	IT	Noida	₹58,000	10-Jan-2022	5	Data Analyst
119	Manish Bajpai	IT	Noida	₹52,000	19-Jul-2021	3	QA Engineer
106	Anjali Mishra	Marketing	Agra	₹61,000	30-Nov-2020	4	Marketing Lead
116	Kavya Rao	Marketing	Agra	₹45,000	11-May-2021	3	SEO Specialist
112	Simran Kaur	Marketing	Agra	₹35,000	03-Apr-2023	2	Content Writer
108	Neha Pandey	Sales	Lucknow	₹80,000	18-Jun-2017	5	Sales Manager
117	Sanjay Dubey	Sales	Lucknow	₹43,000	17-Feb-2020	2	Sales Executive
102	Priya Verma	Sales	Lucknow	₹42,000	01-Jul-2020	3	Sales Executive
113	Deepak Joshi	Sales	Lucknow	₹40,000	01-Dec-2021	3	Sales Executive
120	Tanya Kapoor	Unassigned	Delhi	₹15,000	08-Jan-2024	2	Intern

Filtered result: **Department = IT** and **Salary > 55,000** (5 of 20 records shown).

Task 6 - Filtered View: Department = IT AND Salary > 55,000

Filter applied through the AutoFilter drop-downs (Department = IT, Salary > 55000). Non-matching rows are hidden.

Emp ID	Employee Name	Department	City	Salary (INR)	Join Date	Rating (1-5)	Job Title
111	Rahul Saxena	IT	Noida	₹85,000	09-Oct-2019	5	Data Scientist
107	Karan Tiwari	IT	Noida	₹70,000	25-Feb-2018	5	DevOps Engineer
101	Aarav Sharma	IT	Noida	₹65,000	15-Mar-2021	4	Software Engineer
115	Arjun Mehta	IT	Noida	₹62,000	21-Jun-2022	4	Software Engineer
103	Rohan Gupta	IT	Noida	₹58,000	10-Jan-2022	5	Data Analyst

Task 7: Conditional Formatting & Excel Functions

Conditional formatting: salary >= 70,000 green, salary < 40,000 red; rating >= 4 green, <= 2 red; performance label coloured by text. **IF()** assigns a performance grade and a 10% bonus for rating >= 4. **COUNTIF()** counts employees per department. **SUMIF()** totals salary per department. The bonus % is an editable input cell.

Task 7 - Conditional Formatting & Functions (IF, COUNTIF, SUMIF)

Formula columns H-I use IF(); summary block on the right uses COUNTIF() and SUMIF(). Bonus % is an editable assumption (blue).

Emp ID	Employee Name	Department	City	Salary (INR)	Join Date	Rating (1-5)	Performance (IF)	Bonus (IF)
101	Aarav Sharma	IT	Noida	₹65,000	15-Mar-2021	4	Excellent	₹6,500
102	Priya Verma	Sales	Lucknow	₹42,000	01-Jul-2020	3	Good	-
103	Rohan Gupta	IT	Noida	₹58,000	10-Jan-2022	5	Excellent	₹5,800
104	Sneha Singh	HR	Kanpur	₹72,000	20-May-2019	4	Excellent	₹7,200
105	Vikram Yadav	Finance	Delhi	₹48,000	12-Sep-2021	3	Good	-
106	Anjali Mishra	Marketing	Agra	₹61,000	30-Nov-2020	4	Excellent	₹6,100
107	Karan Tiwari	IT	Noida	₹70,000	25-Feb-2018	5	Excellent	₹7,000
108	Neha Pandey	Sales	Lucknow	₹80,000	18-Jun-2017	5	Excellent	₹8,000
109	Amit Kumar	Finance	Delhi	₹55,000	05-Aug-2022	3	Good	-
110	Pooja Chauhan	HR	Kanpur	₹38,000	16-Jan-2023	2	Needs Improvement	-
111	Rahul Saxena	IT	Noida	₹85,000	09-Oct-2019	5	Excellent	₹8,500
112	Simran Kaur	Marketing	Agra	₹35,000	03-Apr-2023	2	Needs Improvement	-
113	Deepak Joshi	Sales	Lucknow	₹40,000	01-Dec-2021	3	Good	-
114	Meera Nair	Finance	Delhi	₹90,000	14-Mar-2016	4	Excellent	₹9,000
115	Arjun Mehta	IT	Noida	₹62,000	21-Jun-2022	4	Excellent	₹6,200
116	Kavya Rao	Marketing	Agra	₹45,000	11-May-2021	3	Good	-
117	Sanjay Dubey	Sales	Lucknow	₹43,000	17-Feb-2020	2	Needs Improvement	-
118	Ritu Agarwal	HR	Kanpur	₹41,000	28-Nov-2022	4	Excellent	₹4,100
119	Manish Bajpai	IT	Noida	₹52,000	19-Jul-2021	3	Good	-
120	Tanya Kapoor	Unassigned	Delhi	₹15,000	08-Jan-2024	2	Needs Improvement	-

Assumptions / Legend

Colour rules

Bonus % for Rating >= 4

10%

Salary >= 70,000 = green; < 40,000 = red

Rating >= 4 green, <= 2 red

Department	COUNTIF (Employees)	SUMIF (Total Salary)
Sales	4	₹205,000
IT	6	₹392,000
HR	3	₹151,000
Finance	3	₹193,000
Marketing	3	₹141,000
Unassigned	1	₹15,000
Total	20	₹1,097,000

Other examples

Employees with Rating >= 4	10
Employees with Salary > 60,000	8
Total salary of Rating >= 4 staff	₹684,000
Total bonus payable	₹68,400
Check: total = dataset salary sum	OK - matches

Performance : =IF(G5>=4, "Excellent", IF(G5>=3, "Good", "Needs Improvement"))
 Bonus : =IF (G5>=4, E5*\$L\$5, 0)
 COUNTIF : =COUNTIF (\$C\$5:\$C\$24, K10)
 SUMIF : =SUMIF (\$C\$5:\$C\$24, K10, \$E\$5:\$E\$24)

Total of the SUMIF column (Rs. 10,97,000) equals the SQL SUM(salary) result from Task 2.6 - the two tools agree.

Task 8: VLOOKUP & XLOOKUP

Two related tables are joined on **Dept ID**: Employees and Departments. **VLOOKUP** fetches Dept Name and Location; **XLOOKUP** fetches the Manager. Employee 120 has no department, so both return 'Not Assigned'. The lower block shows the key difference: XLOOKUP can look *left* (Name -> Emp ID), VLOOKUP cannot.

Task 8 - VLOOKUP & XLOOKUP across two related tables

Table 1 (A-C) = Employees, Table 2 (H-K) = Departments. Common column = Dept ID.

Emp ID	Employee Name	Dept ID	Dept Name (VLOOKUP)	Location (VLOOKUP col 4)	Manager (XLOOKUP)
101	Aarav Sharma	2	IT	Noida	Rahul Saxena
102	Priya Verma	1	Sales	Lucknow	Neha Pandey
103	Rohan Gupta	2	IT	Noida	Rahul Saxena
104	Sneha Singh	3	HR	Kanpur	Sneha Singh
105	Vikram Yadav	4	Finance	Delhi	Meera Nair
106	Anjali Mishra	5	Marketing	Agra	Anjali Mishra
107	Karan Tiwari	2	IT	Noida	Rahul Saxena
108	Neha Pandey	1	Sales	Lucknow	Neha Pandey
109	Amit Kumar	4	Finance	Delhi	Meera Nair
110	Pooja Chauhan	3	HR	Kanpur	Sneha Singh
111	Rahul Saxena	2	IT	Noida	Rahul Saxena
112	Simran Kaur	5	Marketing	Agra	Anjali Mishra
113	Deepak Joshi	1	Sales	Lucknow	Neha Pandey
114	Meera Nair	4	Finance	Delhi	Meera Nair
115	Arjun Mehta	2	IT	Noida	Rahul Saxena
116	Kavya Rao	5	Marketing	Agra	Anjali Mishra
117	Sanjay Dubey	1	Sales	Lucknow	Neha Pandey
118	Ritu Agarwal	3	HR	Kanpur	Sneha Singh
119	Manish Bajpai	2	IT	Noida	Rahul Saxena
120	Tanya Kapoor		Not Assigned	-	Not Assigned

Dept ID	Dept Name	Manager	Location
1	Sales	Neha Pandey	Lucknow
2	IT	Rahul Saxena	Noida
3	HR	Sneha Singh	Kanpur
4	Finance	Meera Nair	Delhi
5	Marketing	Anjali Mishra	Agra
6	Legal	Adv. R. Malhotra	Delhi

Where **XLOOKUP** is different: look-up to the **LEFT** (find Emp ID from a Name)

Search name: **Rohan Gupta**

VLOOKUP result: **Not found - VLOOKUP cannot look left**

XLOOKUP result: **103**

VLOOKUP always searches the **FIRST** column of the range, so Name -> Emp ID (a column to its left) is impossible.

XLOOKUP takes separate lookup and return arrays, so it can return from any side.

Feature	VLOOKUP	XLOOKUP
Syntax	VLOOKUP(value, table, col_index, FALSE)	XLOOKUP(value, lookup_array, return_array, [not_found])
Look left?	No	Yes
Default match	Approximate (needs FALSE)	Exact
Column insert breaks it?	Yes (col_index number)	No (direct range)
Not-found message	Needs IFERROR wrapper	Built-in 4th argument
Availability	All Excel versions	Excel 2021 / Microsoft 365

VLOOKUP : =IFERROR(VLOOKUP(C5,\$H\$5:\$K\$10,2,FALSE),"Not Assigned")

XLOOKUP : =XLOOKUP(C5,\$H\$5:\$H\$10,\$J\$5:\$J\$10,"Not Assigned")

Left look-up : =XLOOKUP(B28,B5:B24,A5:A24,"Not found")

Task 9: Pivot Table & Charts

Source: 60 sales transactions (Jan-Jun 2026) with Region, Category, Product, Quantity, Unit Price and Revenue (= Quantity x Unit Price). The summary shows **Rows = Region, Columns = Category, Values = Sum of Revenue**, plus a product-wise summary.

Task 9 - Pivot-style Summary: Revenue by Region x Category

Rows = Region, Columns = Category, Values = Sum of Revenue (SUMIFS on Task9_SalesData). Built with formulas so it updates automatically.

Region	Accessories	Electronics	Furniture	Grand Total
North	₹38,500	₹820,000	₹37,500	₹896,000
South	₹69,700	₹510,000	₹136,500	₹716,200
East	₹46,000	₹28,000	₹37,500	₹111,500
West	₹40,900	₹649,000	₹19,500	₹709,400
Grand Total	₹195,100	₹2,007,000	₹231,000	₹2,433,100

Check: pivot total = data total

OK

Product	Units Sold	Revenue	% of Total
Laptop	25	₹1,375,000	56.5%
Smartphone	23	₹506,000	20.8%
Monitor	9	₹126,000	5.2%
Headphones	58	₹104,400	4.3%
Keyboard	47	₹56,400	2.3%
Mouse	49	₹34,300	1.4%
Office Chair	18	₹135,000	5.5%
Desk	8	₹96,000	3.9%

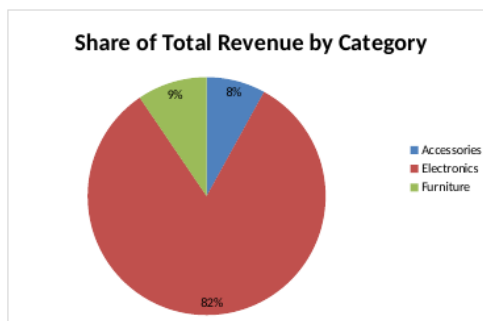
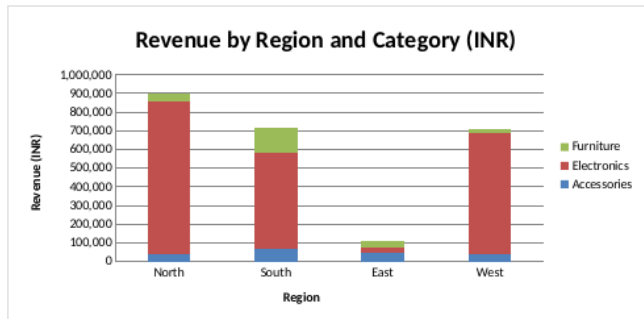


Chart explanation: The stacked column chart shows North has the highest revenue (Rs. 8.96 lakh), followed by South and West, while East is far behind (Rs. 1.12 lakh). In every region **Electronics** is the largest part, and the pie chart confirms it contributes about 82% of total revenue (Rs. 24.33 lakh), so stock and marketing effort on Electronics, especially laptops (56.5% of all revenue), matters most. East is the weakest region and is the clear growth opportunity.

Sales data used for the Pivot Table (first page)

Task 9 - Sales Transactions (source data for the Pivot Table)

Order ID	Date	Region	Category	Product	Quantity	Unit Price (INR)	Revenue (INR)
ORD1001	12-Mar-2026	South	Accessories	Headphones	5	₹1,800	₹9,000
ORD1002	23-Jun-2026	West	Electronics	Laptop	1	₹55,000	₹55,000
ORD1003	25-Feb-2026	North	Electronics	Laptop	2	₹55,000	₹110,000
ORD1004	16-Jun-2026	West	Accessories	Mouse	10	₹700	₹7,000
ORD1005	13-Mar-2026	South	Electronics	Monitor	1	₹14,000	₹14,000
ORD1006	13-Mar-2026	South	Accessories	Keyboard	4	₹1,200	₹4,800
ORD1007	27-Jan-2026	West	Furniture	Desk	1	₹12,000	₹12,000
ORD1008	04-Jun-2026	North	Electronics	Smartphone	3	₹22,000	₹66,000
ORD1009	01-Feb-2026	North	Accessories	Mouse	8	₹700	₹5,600
ORD1010	08-Jun-2026	South	Accessories	Mouse	7	₹700	₹4,900
ORD1011	19-Jun-2026	East	Accessories	Headphones	5	₹1,800	₹9,000
ORD1012	26-Jan-2026	East	Furniture	Office Chair	4	₹7,500	₹30,000
ORD1013	11-Feb-2026	East	Accessories	Keyboard	7	₹1,200	₹8,400
ORD1014	29-Jun-2026	South	Electronics	Smartphone	1	₹22,000	₹22,000
ORD1015	11-Feb-2026	West	Accessories	Headphones	9	₹1,800	₹16,200
ORD1016	26-Jun-2026	East	Electronics	Monitor	2	₹14,000	₹28,000
ORD1017	28-Feb-2026	East	Furniture	Office Chair	1	₹7,500	₹7,500
ORD1018	24-Feb-2026	South	Accessories	Headphones	7	₹1,800	₹12,600
ORD1019	14-Jun-2026	South	Accessories	Keyboard	9	₹1,200	₹10,800
ORD1020	24-May-2026	West	Electronics	Laptop	3	₹55,000	₹165,000
ORD1021	03-Apr-2026	South	Furniture	Desk	2	₹12,000	₹24,000
ORD1022	13-Jan-2026	South	Accessories	Headphones	3	₹1,800	₹5,400
ORD1023	19-Apr-2026	West	Accessories	Mouse	3	₹700	₹2,100
ORD1024	16-May-2026	North	Accessories	Keyboard	6	₹1,200	₹7,200
ORD1025	24-Jun-2026	East	Accessories	Headphones	10	₹1,800	₹18,000
ORD1026	29-Jan-2026	West	Accessories	Keyboard	6	₹1,200	₹7,200
ORD1027	09-Mar-2026	North	Electronics	Laptop	2	₹55,000	₹110,000

Submission Checklist & Files

- **Week3_SQL_Queries.sql** - schema, sample data and every query from Tasks 1-5.
- **Week3_Excel_Workbook.xlsx** - workbook for Tasks 6-9.
- **Week3_SQL_Excel_Report.pdf** - this report.
- Google Drive link: [paste link] - set sharing to *Anyone with the link* → *Viewer*.